

Coherent Domain Adaptation

Transfer of general LLM knowledge to domain-specific learning

Gary Nan Tie, July 15, 2026

Abstract

For a LLM pre-trained on a large generic corpus, we introduce a coherent way to transfer this general knowledge to training a domain-specific LLM.

Introduction

A pre-trained implicit model is viewed as defining a dynamical system. During domain adaptation, instead of merely initializing parameters from the pre-trained model, we constrain the new implicit model to reproduce the hidden equilibria and outputs of the base model on the shared vocabulary while remaining free to learn new dynamics on domain-specific data.

Preliminaries

input pxm matrix $U = [u_1, \dots, u_m]$

(an m-long token string)

output qxm matrix $Y = [y_1, \dots, y_m]$

An **implicit model** [4] consists of equilibrium equation (E)
in fixed point state nxm matrix X and prediction equation (P):

$$X = \phi(AX + BU)$$

$$Y^{\wedge}(U) = CX + DU$$

where $\phi : \mathbb{R}^{n \times m} \rightarrow \mathbb{R}^{n \times m}$ is a strictly increasing nonlinear activation function and matrix A is parameterized [5] as $A = T - \text{diag}(|T| \mathbf{1}_n) + \delta \text{Id}_n$, $0 < \delta < 1$, to ensure fixed points exist and are unique.

We now consider a pair of implicit models, base-line and domain-specific, and show how general knowledge learned from the former can be transferred to the domain specific latter.

A token is a column vector. Let U_b denote base-line input tokens and let U_d denote domain-specific input tokens.

Suppose token intersection $U := U_b \cap U_d$ is non-empty.

Base-line implicit model:

$$X = \phi(A_b X + B_b U_b)$$

$$Y^{\wedge}(U_b) = C_b X + D_b U_b$$

Restricted to U (a subset of U_b),

$$X_b = \phi(A_b X_b + B_b U) =: \phi(Z_b)$$

$$Y^{\wedge}_b(U) = C_b X_b + D_b U$$

Domain-specific implicit model:

$$X = \phi(AX + BU_d)$$

$$Y^{\wedge}(U_d) = CX + DU_d$$

Constrained implicit model (U a subset of U_d):

$$X = \phi(AX + BU)$$

$$Y^{\wedge}(U) = CX + DU$$

subject to constraints [4],

$$Z_b = AX + BU$$

$$Y^{\wedge}_b = CX + DU$$

Note that $Z_b = AX + BU$ and $\phi(Z_b) = X_b$
implies $X = \phi(AX + BU) = \phi(Z_b) = X_b$

So the constraints imply $X = X_b$ and $Y^* = Y_b^*$,
that is, both the equilibrium state and output
of the constrained domain-specific implicit model
match those of the base-line implicit model.

We then solve the training problem:

minimize $L(A, B, C, D)$ over A, B, C, D

subject to

$$Z_b = AX + BU$$

$$Y_b^* = CX + DU$$

where L is a user-defined loss function.

Through these training constraints, this is how implicit models transfer general knowledge to domain-specific ones.

Implicit recurrent LLM [1]

We will apply this technique to the implicit recurrent models defining LLM **AliceCIA**, which we now recall.

Let $\text{Emb: Token} \rightarrow \text{Vector}$ be a word embedding,
and let $v_i = \text{Emb}(t_i)$.

Denote a LLM by a map $\text{Token} \rightarrow \text{Continuation}$.

Let $\theta = \{ (W_1, b_1), (W_2, b_2) \}$ be parameters
for a **feedforward network** defined by
 $\text{FFN}_\theta(h_i) := \text{ReLU}(h_i W_1 + b_1) W_2 + b_2$

A **state machine** (S, O) begins in an initial state h_0 ,
that gets updated with each successive input t_i ,

via $h_i = S(h_{i-1}, t_i)$ where S is a transition function, and then outputs $o_{i+1} = O(h_i)$, where O is an output function. (The right index shift is for autoregressive continuation.) Diagrammatically, picture a state machine as a spine with vertebrae hidden states h and ribs either side t - h - o , token to state to output.

implicitRNN [4] functions similarly to a vanilla RNN, processing sequence inputs one by one. For each time step i , where $t_i \in \mathbb{R}^p$ represents the i -th token in a sequence $(t_1, t_2, \dots, t_m)$, the input u_i for implicitRNN is a concatenation of t_i and the previous hidden state h_{i-1} , akin to a vanilla RNN. The implicit prediction rule for implicitRNN is as follows:

$$h_0 = 0, \quad k_0 = 0$$

$$u_i = [t_i \ h_{i-1}]'$$

$$k_i = \phi(Ak_i + Bu_i) \text{ (equilibrium equation)}$$

$$h_i = S(h_{i-1}, t_i) := Ck_i + Du_i \text{ (prediction equation)}$$

where $A := \Gamma - \text{diag}(|\Gamma| \mathbf{1}) + \gamma \mathbf{Id}$, $0 < \gamma < 1$, [5] ensures A well-posed for ϕ , that is, the fixed point equation has a solution and it is unique.

In this setup, the recurrent layer is replaced with an implicit structure comprising the equilibrium and prediction equations, where we learn matrices A, B, C, D and fixed points k_i . Note that h_i is a function of k_i .

Alice [1]

Is a state machine with transition and output functions (S, O) , where S is defined by the implicitRNN above, and hidden state h_i is viewed as a contextual representation of token t_i .

Given a token sequence $t_1, t_2, \dots, t_i$ the goal is to find a token continuation o_{i+1} with certain properties. The state machine output enables this by choosing the continuation o_{i+1} to be $O(h_i)$, defined by attention layers with the desired (perplexity and Renyi α -entropy) properties.

Attention layer

Input h_i .

Let score be a desired similarity function, like a reproducing kernel.

For $i \in [m]$, define probabilities

$\alpha_{ij} := \text{soft max over } j \in [i] \text{ score}(h_i, k_j)$.

Let $h_i^\wedge = \sum_{j \in [i]} \alpha_{ij} v_j$, where $v_j = \text{Emb}(t_j)$,

and define update $h_i' := \text{FFN}\Theta(h_i^\wedge)$.

Output h_i' .

Stack L attention layers $\Theta = (\theta_1, \dots, \theta_L)$ composing updates and now let h_i' denote the final L -th update.

For $i \in [m]$, define probabilities

$\beta_{ij} := \text{soft max over } j \in [i] \text{ score}(h_i', k_j)$

and let $j^*i := \arg \max_{j \in [i]} \beta_{ij}$.

Define token continuation $o_{i+1} = \mathbf{O}(h_i) := t^{j^*i}$,

the token whose fixed point k to date is most similar
(probable under distribution β) to update h' .

Define the average Renyi α -entropy of $\{t_1, \dots, t_m\}$,

a measure of semantic alignment by:

$\mathbf{E}(\{t_i\}) := (1/m) \sum_i H(i, \alpha)$,

where $H(i, \alpha) := H_\alpha(\beta_{ij})$, the Renyi α -entropy of $\{\beta_{ij}\}$.

Together perplexity of output $\mathbf{O}$ and average Renyi α -entropy $\mathbf{E}$ train the h_i to holistically capture syntax and semantics.

Alice training problem

Let $\lambda > 0$, learn implicitRNN matrices A, B, C, D and fixed points k_i for h_i , as well as, feedforward network parameters Θ to minimize:

$[\text{Perplexity}(o_2, \dots, o_{m+1}) + \lambda \mathbf{E}(\{t_i\})]$

Remarks:

$\mathbf{O}$ next-token continuation $t_1, \dots, t_i$, o_{i+1} reflects the word embedding through the $h_i^\wedge$.

Perplexity intrinsically evaluates the continuation,

regularized by average Renyi α -entropy.

Solving the Alice training problem, is solving the **next-token learning** problem jointly constrained by syntactic co-occurrence (perplexity) and semantic alignment (Renyi α -entropy).

Thus an Alice state h_i results in output $o(h_i)$, a token continuation, that defines a LLM.

Ecila

is a sister state machine **(S, O')**, where $o'_i = \mathbf{O}'(h_i) := \text{FFN}\theta'(h_i)$ approximates $v_i := \text{Emb}(t_i)$

In detail, the **Ecila training problem**

Learn implicitRNN matrices A, B, C, D and fixed points k_i for h_i , and feedforward network parameters θ' for o'_i to minimize:

$$\sum_{i \in \{m\}} 1 - \text{Cos}(o'_i, v_i)$$

So **Ecila** simulates Emb, the word embedding.

AliceCila $_{\tau}$

Is the state machine **(S, O, O')** with two output functions, mother to embedding Ecila and transformer sister Alice.

The **AliceCila $_{\tau}$ training problem**

Given $\lambda > 0$, for $\tau \in [0,1]$ learn matrices A, B, C, D and fixed points k_i for h_i , parameters Θ and θ' , to minimize:

$$(1 - \tau) \sum_{i \in \{m\}} 1 - \text{Cos}(o'_i, v_i) + \tau [\text{Perplexity}(o_2, \dots, o_{m+1}) + \lambda E(\{t_i\})]$$

So 'word embedding' Ecila(S, O') is deformed by AliceCila(S, O, O') $_{\tau}$ to 'LLM' Alice(S,O), in the sense AliceCila $_{\tau}$ states h_i deform from Ecila's states to Alice's states.

This also defines a map from Word Embedding to LLM, given by (Emb, Alice).

And back

Given Alice(S, O), a function of Emb, $oi+1 := O(hi)$.

Note that the function, ti to Emb($oi+1$) defines a new embedding Emb', and (Alice, Emb') defines a map LLM to Word Embedding.

Unrolling this generates a chain: Emb, Alice, Emb', Alice', Emb'', Alice'', ...

From the purposes of token continuation, we have many choices, which Emb-Alice pair iterate, and which $\tau \in [0,1]$ for AliceciIA_ τ whose output O determines a token continuation. Thus AliceciIA_ τ is a fifth strategy [2] for learning continuations [in addition to joint and staged learning of O, from Alice(S,O) and AliceciIA(S, O, O')].

More generally, note that deformation [3] is a meta-technique from encoder to predictor.

Knowledge transfer

Returning to knowledge transfer, suppose LLM **AliceciIA**_ τ has been pre-trained on a large general corpus and we now wish to learn it on specialized domain-specific data like legal, medical or finance documents, while retaining the hard won general language knowledge, without having to relearn it from technical jargon. We can achieve this by adding state (fixed points k) and output (contextual token representations h) constraints from the pre-trained general base-line implicit recurrent model to the AliceciIA_ τ (S, O, O') training problem (a function of A, B, C, D), when learning over domain-specific data. Akin to updating a prior, we learn a domain-specific LLM leveraging general language knowledge by applying a constrained implicit model. One does not have to start all over again learning from scratch, when learning in new domains! LLM **AliceciIA**, an implicit state machine, enables coherent knowledge transfer from general to specific.

References

[1] 'Alice Through the Word Embedding'

DOI: 10.13140/RG.2.2.34901.79841

https://www.researchgate.net/publication/408460384_Alice_Through_the_Word_Embedding?channel=doi&linkId=6a4921a95ec65810298eecaf&showFulltext=true

[2] 'Strategies for learning token continuations'

DOI: 10.13140/RG.2.2.16884.31366

[3] 'State deformation from word embedding to LLM'

DOI: 10.13140/RG.2.2.26347.71204

[4] 'The Extrapolation Power of Implicit Models'

Juliette Decugis, Alicia Y. Tsai,

Max Emerling, Ashwin Ganesh, Laurent El Ghaoui

arXiv:2407.14430v1 [cs.LG] 19 Jul 2024

[5] 'Robust Implicit Networks via Non-Euclidean Contractions'

Saber Jafarpour, Alexander Davydov,

Anton V. Proskurnikov, Francesco Bullo

arXiv:2106.03194v6 [cs.LG] 26 Jan 2022